

CorpusMind Voice

Page 1 of 2 — Setup, recording and the six-step pipeline

v1.1.0 · 100% offline

Local-first audio → corpus

CorpusMind Voice is the audio companion of [CorpusMind](#), the local-first research environment for corpus linguistics and multimodal discourse analysis. It converts recordings (MP3, MP4, M4A, AAC, OGG, OPUS, WAV, FLAC, WEBM, WMA, AMR, 3GP, AIFF) or live microphone input into a linguistically annotated corpus on your own machine, and analyses it academically. No account is needed, nothing is uploaded, and no cloud API is ever contacted — the privacy model is absolute by design. This two-page guide walks you through installation, the six-stage pipeline, and the editorial workflow; terminology follows the conventions used on the project site.

1 Installation

The web build runs anywhere Node.js (or Bun) runs: `bun install`, `bun run db:push`, then `bun run dev` and open `http://localhost:3000`. Installable as a **PWA** from your browser's address bar (or the in-app *Install app* button) — on a phone or tablet the PWA lets you **record and analyse on the go**, entirely on-device. Native **desktop builds** for Windows (.exe / .msi), macOS (Apple Silicon & Intel .dmg) and Linux (.deb / .AppImage) are attached to each GitHub release; they bundle their own server and need no Python installed. For research-grade output also install the Python worker: `pip install -r python/requirements.txt` (faster-whisper, ParSelmouth). Montreal Forced Aligner is optional and adds phoneme-grade boundaries (`conda install -c conda-forge montreal-forced-aligner`, then download the `arabic_mfa` / `english_us_arpa` acoustic models and dictionaries).

A first-launch **Welcome window** (three short pages, English/Arabic) introduces the app, the offline privacy model and a quick start; a **Light / Dark / System theme** toggle lives in the header.

2 Models, packages and hardware

Open **Settings and Diagnostics** before your first job.

- **Models & packages** — download any Whisper size on demand: `tiny` (75 MB), `base` (145 MB), `small` (484 MB), `medium` (1.5 GB) or `large-v3` (3.1 GB, recommended). Models are stored in the app's private data folder, survive restarts and updates, and are reused offline; downloads are resumable and show live progress. Delete a model to reclaim space at any time.
- **Hardware & Diagnostics** — the panel reads your CPU, RAM and — via `nvidia-smi` — your NVIDIA GPU. With CUDA present, ASR runs INT8-float16 on the GPU; without it, the app explicitly warns that processing falls back to CPU (INT8) and can take up to $\approx 2 \times$ real time. You can also pin the device per job in the **Studio** tab (Auto / CPU / GPU) and pick the Whisper model per job.
- **Local LLM** — Ollama (port 11434) and LM Studio (port 1234) are detected automatically exactly like the parent CorpusMind (127.0.0.1 + `OLLAMA_HOST`); on desktop the app can start `ollama serve` for you.

3 Record and process (the six steps)

1. In **Studio**, drop a file or click the drop zone to browse; or click **Record from microphone** to capture live speech (press *Stop recording* to upload the take). On desktop the app requests your OS microphone permission once (macOS) or grants it automatically (Windows); on mobile browsers a codec fallback chain (webm/opus → mp4/aac → ogg) keeps recording working everywhere. 2. Pick the **language** (English, Arabic — Egyptian vernacular عامية, Arabic — MSA فصحي), the **compute device** and the **Whisper model**. 3. Click **Start pipeline**. Progress is shown per stage, streamed live from the local task queue:

#	Stage	What happens
1	Ingest & preprocess	container decoded to 16 kHz mono PCM WAV (PyAV/pydub)
2	ASR	faster-whisper (chosen model) INT8; word timestamps + confidence
3	Forced alignment	MFA refines word/phoneme boundaries (when installed)
4	Prosody	Praat via ParSelmouth: F0 (50–400 Hz), intensity, jitter, shimmer, HNR
5	Disfluency	pauses > 200 ms, fillers (um, uh, إيه, آه, يعني), repeats, false starts, interruptions, lengthenings
6	Structured output	SQLite (3 tables) + JSON + TEI/XML written locally

If the Python worker is not installed, the app runs its **simulation engine** instead — the same six stages with a demo corpus — and labels the job *Simulation* so demo data is never mistaken for real transcriptions.

4 Correct the transcript (confidence-coded editor)

Open the **Transcript Editor** tab. Every token carries the ASR confidence as a background colour: **green** ≥ 0.85 , **amber** $0.60\text{--}0.85$, **red** < 0.60 — skim the red tokens first. Click any token to correct it: saving marks the token *edited* and queues the containing utterance for a **single-utterance re-alignment pass**, so a one-word fix never re-processes the whole file. Each utterance also exposes its detected disfluencies (pause counts, fillers, repeats, false starts, interruptions, lengthenings) and prosody summary (mean/min/max F0, intensity, jitter, shimmer, HNR). A **Save to device** row at the bottom of the tab writes JSON / CSV / TEI/XML / SQLite whenever you need it.

5 Linguistic analysis (new in v1.1.0)

The **Linguistic Analysis** tab computes a full academic report locally — nothing is uploaded. Eight modules:

Overview — tokens, types, TTR, speech rate (wpm), utterances, hapax legomena, confidence distribution.	Frequency — word list with counts, per-1,000 rates, Gries DP dispersion , function-word filter, bar chart.
KWIC — concordancer with configurable context window, case toggle, timestamps; exportable.	Keywords — log-likelihood G² and LogRatio effect size (Hardie 2014) against your other sessions, or a built-in function-word reference.
Collocations & N-grams — bigram association (MI , t-score , logDice) plus top bigram/trigram/4-gram lists.	Lexical diversity — length-robust MATTR (window 50) and MTLD (≥ 0.72), plus plain TTR and lexical density .
Readability — Flesch Reading Ease and Flesch–Kincaid grade (English), language-neutral LIX and long-word share.	Speech & Prosody — disfluency rates per 1,000 tokens by type, pause statistics, Praat aggregates (F0 mean/range, intensity, jitter, shimmer, HNR).

Every table saves to CSV, and the whole report saves as JSON.

6 Corpus metadata (TEI header)

In **Metadata** describe the session: title, speaker name or pseudonym, dialect, gender, age, recording date and place, genre, licence and free notes. These fields are embedded into the `<teiHeader>` of every saved file, exactly the fields CorpusMind expects when importing — see the metadata chapter on the project site. The tab also carries the **Save to device** row (JSON / CSV / TEI/XML / SQLite). Arabic metadata is fully supported (the whole UI switches to right-to-left Arabic with the Cairo typeface from the العربية toggle). On the machine where CorpusMind is installed, the **Launch CorpusMind** button in Settings and Diagnostics opens the companion app with this corpus ready to query.

7 Local AI assistant and tools (optional)

The **Assistant** tab chats with a *local* model through **Ollama** (<http://127.0.0.1:11434>) or **LM Studio** (<http://127.0.0.1:1234>) — e.g. "list utterances with more than two fillers". Start Ollama once with `ollama serve` and pull any model (`ollama run llama3.1`), or load a model in LM Studio; pick the model in the tab or in Settings. Three per-transcript tools run through the same local model: **Summarise**, **Preview without fillers**, and **Suggest topic tags**. If no provider is reachable, the panel says so and never substitutes a cloud service.

8 Troubleshooting

Symptom	Fix
Job labelled <i>Simulation</i>	Python worker missing → <code>pip install -r python/requirements.txt</code>
<i>MFA skipped</i> in stage 3	Optional: install MFA + dictionaries (see §1)
<i>No NVIDIA GPU detected</i>	Expected on CPU-only machines; INT8 CPU mode is used
Whisper model missing in Studio	Settings and Diagnostics → Models & packages → Download
Microphone button does nothing	Grant mic permission in the browser/OS, or upload a file; recording needs HTTPS or localhost
Assistant unreachable	Start <code>ollama serve</code> or load a model in LM Studio (port 1234), then re-open the tab
Downloaded model "vanished"	It cannot — models persist in the app data folder; check the path shown in Settings